

Product Spec

Opportunity

Search in HealthSource, the core function that enables users to locate and verify medical request data, is slow and inconsistent. It erodes user trust, introduces operational drag, and impacts time-sensitive workflows across the business.

This initiative aims to deeply understand **why** search is slow and **what can be done** to make it dramatically faster and more reliable — without changing the user interface or workflows.

Improving the speed and consistency of HealthSource search unlocks:

- Faster response times for internal teams validating and managing requests
- Better client experience with reduced waiting and error handling
- Stronger foundation for future workflow automation and analytics

This is a platform-level investment with outsized downstream impact.

Goal & Impact

Goal: Dramatically reduce average and P95 search response times within HealthSource while maintaining data accuracy and functional parity.

Intended Impact:

- Search latency is no longer a bottleneck for request tracking or case validation
- Engineers have a measurable understanding of performance drivers and can sustain improvements over time
- Internal and client-facing users experience consistent, reliable response times regardless of query complexity

This work should shift perception of HealthSource search from “slow but reliable” to “instant and dependable.”

Hypotheses to Test

1. **Infrastructure Hypothesis:** Search latency is primarily due to inefficient indexing and caching mechanisms, not inherent database limitations. If true, performance can be improved without major rearchitecture.

2. **Application Hypothesis:** Query formulation and result pagination logic contribute meaningfully to performance issues. Optimizing this logic will yield measurable time reductions.
3. **Systemic Hypothesis:** There are hidden dependencies (e.g., service orchestration, logging, or throttling) adding latency. Isolating and refactoring these dependencies will improve consistency across all queries.

Each hypothesis is designed to reveal both immediate optimization opportunities and longer-term architectural constraints.

Press Release (Future State Announcement)

FOR IMMEDIATE RELEASE — HealthSource Search Now Up to 10x Faster

We're excited to announce a major improvement to HealthSource search performance. With this release, users will experience near-instant results when searching for requests, cases, and documents — even across large datasets.

Behind the scenes, we've reengineered how HealthSource indexes, retrieves, and returns information, making the platform not only faster but also more reliable under heavy workloads.

This enhancement reinforces our commitment to building a responsive, resilient platform that scales with our customers' needs — enabling faster decisions, fewer delays, and greater confidence in every search.

Pre-Mortem: How This Could Go Sideways

- **Root Cause Ambiguity:** We spend time optimizing the wrong layer because we didn't isolate true bottlenecks early. *Mitigation:* Begin with clear performance profiling and instrumentation.
- **Incremental Wins Only:** Improvements are measurable but not transformative. *Mitigation:* Set a clear performance bar (e.g., 5x improvement) and require proof before declaring success.
- **Regression Risk:** Optimizations degrade accuracy or reliability of results. *Mitigation:* Maintain parity testing and automated validation suites.
- **Tech Debt Trap:** Fixes create brittle dependencies or shortcuts that slow future development. *Mitigation:* Implement structural improvements only where sustainable; document every change clearly.

Prioritized Scope

In Scope

- Full technical analysis of the search service architecture, indexing, and query execution flow
- Identification and resolution of backend performance bottlenecks
- Benchmarking before/after results and ongoing monitoring instrumentation
- Backend-only improvements (no UI or UX changes)

Out of Scope

- Any visual or workflow modifications
- Search ranking or relevancy adjustments
- Introduction of new data fields or sources

Acceptance Criteria

- HealthSource returns search results within the targeted performance threshold (TBD during alpha).
- No degradation in result accuracy or stability.
- Measurable performance gains validated across standard and complex query types.
- Code-level diagnostics and performance monitoring are in place for ongoing observability.

Feature Type

- **Feature** — Significant backend performance improvement requiring investigation, optimization, and validation.

User-Facing?

- **No.** No visual or workflow changes.

Dependencies & Constraints

- Dependent on backend engineering (search service and database optimization).
- Collaboration with infrastructure for caching and indexing optimization.
- Legacy service dependencies may limit some options for rearchitecture.

Risks & Mitigation

- **Risk:** Improvement limited by underlying database technology.
Mitigation: Identify alternative indexing strategies and long-term architectural path.
- **Risk:** Variability across customer datasets masks improvement.
Mitigation: Test across representative data sets.
- **Risk:** Deployment risk during reindexing.
Mitigation: Use staged rollouts and feature toggles.

Data & Analytics Plan

- Establish baseline and target performance metrics (avg, P95 response times).
- Measure improvements across multiple query categories.
- Integrate real-time monitoring to track latency post-launch.

Release Strategy

- **Alpha:** Internal test with real production datasets to validate hypotheses and measure performance gains.
- **Beta:** Broader internal use; monitor real-world variability and system load.
- **GA:** General release once performance stability and consistency are confirmed.

Tech Spec

Purpose

The technical spec / architectural design document helps align our teams on how we will achieve the Product Spec. There are many ways to solve a problem, and this document describes how we'll balance many competing priorities to achieve a solution that delivers optimal business value.

Section Owner:

The engineer(s) assigned to the discovery and scoping of this feature are responsible for owning the content in this section. We invite others to provide input, feedback, and questions on this section.

Order of Operations

The product and engineering teams collaborate on this document to provide increasing levels of clarity around the requirements and best approach that provides the most business value for each situation.

This process will take several distinct tickets and meetings. The goal is to provide initial visibility to the business about risks, opportunities, and costs. As we hone in on identifying exactly what we build and how much time we have available, the later stages involve finalizing the actual technical design that will be built.

While the whole document is needed for a complete review, a "first pass" sizing should always include:

- Design Goals
- One or more proposed architectural designs
- Size Estimate
- Confidence
- Risks/Mitigations table
- Rollout mechanics

Subsequent passes should increase the detail level (accuracy and precision) of the above sections.

During the final pass discovery stage and prior to the kickoff meeting, all remaining sections should be completed as best as possible.

Note: the kickoff meeting must always include Tyler Beach to help satisfy requirements for CASTLE

(<https://datavant.atlassian.net/wiki/spaces/PS/pages/1508179991/CASTLE+for+Engineers>)

System Context

Why: A simple explanation that lists the system components/domains involved and their current state so that readers can be oriented for the remaining context in the document.

What: 1 or 2 paragraphs describing the state of the system as it pertains to the requested change. Links to any previous architecture diagrams or other artifacts helpful in understanding this area.

Design Goals

Why: In order to help align our teams on business value and tradeoffs, we want to explicitly state our design goals that were prioritized when developing the proposed designs.

What: A paragraph or some bullet points of how we balanced such competing forces as, for example:

- Do we build it ourselves, partner with someone, or buy a solution?
- How do we balance quality, cost, and time so that we achieve an optimal business outcome?
- Is a human powered/manual approach better or does automation yield a better balance for the business (short term vs. long term)?
- Is phasing necessary to minimize risk, and what is the appropriate complexity to try to wrangle in the proposed design?
- How important is developer experience/velocity vs. reusability and maintainability of a given change? What is best for the business right now and in the future?
- We are lacking good test coverage in many places in HealthSource. How risky is the change, and how much risk mitigation is necessary as an implicit part of designing the solution?

Proposed Architectural Designs

Why: To help us align our teams on what the change would actually look like, how simple/complex it is, and what pieces of the system are in play for a given change. This is what we're sizing.

What: A collection of text, system diagrams, states, or flows to show what the system would do if we approached the problem a certain way. The description should be in enough detail that the size and complexity of the change can be ascertained. In earlier phases of our design lifecycle, note the options or tradeoffs in each area so that we can collectively align on competing design goals. In later stages of the design, we do not desire ambiguity in this section and want only a single design solution to recommend.

Size Estimate

Why: In order to figure out the right mix of changes for a quarter and also to figure out who are the right people to make the changes, we need to understand an estimate of total cost.

What: Preferably, an estimated number of engineers for an estimated number of total sprints. If we need specific people or teams involved, note that in the cost since dependencies are important contributors to overall cost. Alternatively, hours or t-shirt sizes are acceptable if this is the most accurate we can be. High accuracy is expensive, but we don't want to be irresponsible by being too imprecise.

Confidence

Why: Every project has a unique risk profile and unique tradeoffs/challenges in order to deliver ideal business outcomes, and so we want to try to capture a confidence measure.

What: One or two sentences describing how confident we are about our size estimate.

Risks/Mitigations

Why: Even with the best design, some risks are always present. Call out any open questions or concerns as well as any risks we are well aware of and how we'll defend against them.

What: A table with a list of risks and what we're going to do to reduce the risk (or how the existing design proposed defends against the risk).

Risk	Mitigation

Rollout Mechanics

Why: Delivering business value can rarely/wisely be accomplished in a single step. We want to understand the complexity and steps needed to deliver value to customers, what is delivered in each phase, and how what might need to be communicated at each step of the process.

What: A series of steps, an outline, or a sequence diagram showing how we'd release the changes, what things might need to be migrated/released in each stage, and how we'd rollout the functionality to users. The focus is on what technical changes are needed and what business value users will be delivered during each step.

Note any important system or functional criteria that should be true before advancing to the next phase.

If rollback is for some reason not possible in a step, note that here so we can understand the implications.

This section is intended to be complementary to the **Release Strategy** tab.

Security Considerations and Implications

Why: We want to make sure that all changes take security implications into account and that the risk of those changes is called out.

What: A paragraph or bullet point list describing the known security risks or places where the change interacts with important data, encryption, or auth. We follow the CASTLE process, and this section is not meant to replace that. Tyler Beach is responsible for assessing overall security risk.

Monitoring, Alerting, and Measuring

Why: All software changes need to be supported by the team, and this means having a good monitoring and alerting strategy for the change. Likewise, we want to make sure we're measuring both platform level metrics as well as business level metrics.

What: A bullet point list of what things we'd measure, where we'd measure it (Datadog, Sigma, or a manual query) and alert on. The severity/response plan of the metric should be noted so that we can understand what things wake people up vs. just are used to track adoption.

Testing

Why: All changes must be tested, but not all changes have the same risk level when developing tests. This section helps us align on how deep we should go on which areas of testing the change.

What: A list of specific tests or strategies that we will use to verify the changes, and if applicable, which parts might be done manually vs. in an automated fashion.

Dependencies

Why: Some changes will require new permissions, access, or important work from other teams in order to be complete. We should note that here.

What: A bullet point list of external dependencies that we will need in order for this project to be successful.

Release Strategy

Release Strategy & Iterations

Maturity Level (for Feature or Major Feature):

- **Experiment** – Small-scale (<5 customers) for early hypothesis testing
- **Alpha** – Initial test with up to 10 customers
- **Beta** – Scaled test with up to 100 customers
- **General Availability (GA)** – Broad release to all customers

Normally, features progress through alpha → beta → GA. Skipping steps requires justification.

Rollout Toggles

- Indicate whether a toggle is needed
- PMs should use toggles unless clearly unnecessary
- Toggles, when needed, should ideally be labeled with ticket numbers so that complexity can be manageable long term for our future selves.

Iterations

- Define phases (alpha, beta, GA)
- Include rough timelines
- Note scope and exclusions for each stage